

Lecture 17

ML-KEM (FIPS 203)

SEC 595: Encrypted Computing

Roadmap

ML-KEM is the NIST standard for post-quantum key establishment. It uses no mathematics beyond what the course has built: the ring of Lecture 3, Module-LWE of Lecture 6, the rounding of Lecture 1. Section 1 explains how parameters are chosen, by costing lattice reduction. Section 2 defines a KEM and fixes the module structure. Section 3 gives the IND-CPA scheme K-PKE inside ML-KEM and its correctness. Section 4 covers compression. Section 5 defines IND-CCA2 for KEMs and the Fujisaki–Okamoto transform that achieves it. Section 6 explains why the NTT of FIPS 203 is incomplete, and Section 7 derives the sizes and describes deployment.

1. Lattice Reduction and Parameter Selection

1.1 Lagrange–Gauss Reduction

Lattice reduction takes a bad basis and returns a good basis of the same lattice: it subtracts multiples of one basis vector from another and swaps them when an earlier vector becomes longer than a later one, until no move helps. The shorter the first output vector, the stronger the algorithm.

Definition 1 (Lagrange–Gauss). Given a basis b_1, b_2 of a lattice in $\mathbb{R}^2$, order it so that $\|b_1\| \leq \|b_2\|$ and set

$$\mu = \frac{\langle b_1, b_2 \rangle}{\langle b_1, b_1 \rangle}, \quad b_2 \leftarrow b_2 - \lfloor \mu \rfloor b_1.$$

If now $\|b_2\| < \|b_1\|$, swap and repeat; otherwise stop.

Each step replaces b_2 by $b_2 - \lfloor \mu \rfloor b_1$, a unimodular change, so the lattice is unchanged. The output's first vector has length $\lambda_1(\mathcal{L})$.

Example 1 (The bad basis repaired). The lattice $2\mathbb{Z} \times 3\mathbb{Z}$ has the good basis $(2, 0), (0, 3)$ and the bad basis $b_1 = (2, 3), b_2 = (4, 9)$ (Lecture 5). Both have determinant 6; the bad one has orthogonality defect $\|b_1\| \|b_2\| / \det(\mathcal{L}) = \sqrt{13} \cdot \sqrt{97} / 6 \approx 5.918$. Run Lagrange–Gauss.

Step 1. $\|b_1\| = \sqrt{13} \leq \sqrt{97} = \|b_2\|$. $\mu = (8 + 27)/13 = 35/13 \approx 2.692$, $\lfloor \mu \rfloor = 3$, and $b_2 \leftarrow (4, 9) - 3(2, 3) = (-2, 0)$. Since $2 < \sqrt{13}$, swap: $b_1 = (-2, 0), b_2 = (2, 3)$.

Step 2. $\mu = -4/4 = -1$, $\lfloor \mu \rfloor = -1$, and $b_2 \leftarrow (2, 3) + (-2, 0) = (0, 3)$. Now $\|b_2\| = 3 \geq 2$: stop.

Check. The output $(-2, 0), (0, 3)$ is the good basis up to sign: inner product 0, determinant $|(-2)(3) - 0 \cdot 0| = 6$ ✓, defect $2 \cdot 3/6 = 1$. The first vector gives $\lambda_1 = 2$. Two steps took the defect from 5.918 to 1.

1.2 LLL, BKZ and the Root Hermite Factor

Definition 2 (Root Hermite factor). A reduction algorithm achieves root Hermite factor δ_0 on a d -dimensional lattice if its first output vector satisfies $\|b_1\| = \delta_0^d \cdot \det(\mathcal{L})^{1/d}$. Smaller is stronger.

LLL (Lenstra, Lenstra, Lovász, 1982) extends Lagrange–Gauss to dimension d . It runs in polynomial time and guarantees $\|b_1\| \leq 2^{(d-1)/2} \lambda_1(\mathcal{L})$ and $\|b_1\| \leq 2^{(d-1)/4} \det(\mathcal{L})^{1/d}$, i.e. $\delta_0 \leq 2^{1/4} \approx 1.1892$; in practice it achieves $\delta_0 \approx 1.0219$.

BKZ with block size β runs LLL and then repeatedly solves exact SVP in projected sublattices of dimension β ; $\beta = 2$ is LLL and $\beta = d$ is exact SVP. One SVP call costs $2^{0.292\beta}$ classically and $2^{0.265\beta}$ quantumly (the *core-SVP* cost), and the quality reached is estimated by

$$\delta_0(\beta) \approx \left(\frac{\beta}{2\pi e} (\pi\beta)^{1/\beta} \right)^{1/(2(\beta-1))}.$$

Example 2 (Two block sizes). $\delta_0(100) \approx 1.00926$ at classical cost $2^{0.292 \cdot 100} = 2^{29.2}$, and $\delta_0(400) \approx 1.00398$ at $2^{116.8}$ (quantumly $2^{26.5}$ and $2^{106.0}$). Improving δ_0 in the third decimal place costs a factor of about $2^{87.6}$.

1.3 The Parameter-Selection Loop

Definition 3 (Parameter selection).

1. Fix the scheme. This determines a q -ary lattice (Lecture 5, §6) of dimension d carrying a planted short vector (the secret or the error) of norm about $\sigma\sqrt{d}$.
2. Check that the planted vector is shorter than the Gaussian-heuristic length $\text{gh}(\mathcal{L})$, where $\text{gh}(\mathcal{L})^2 = (d/2\pi e) \det(\mathcal{L})^{2/d}$.
3. Find the δ_0 that BKZ needs to reach the planted vector.
4. Invert $\delta_0(\beta)$ for β ; the cost is $2^{0.292\beta}$.
5. If the cost is below 2^λ , raise d (n or k), lower q , or raise σ , and repeat.

Every parameter table in this course is the output of this loop.

Example 3 (Costing ML-KEM-512). *Target.* For 128 bits classical we need $0.292\beta \geq 128$, i.e. $\beta \geq 438.36$, so $\beta = 439$ and

$$\delta_0(439) = \left(\frac{439}{2\pi e} (439\pi)^{1/439} \right)^{1/876} \approx 1.003732,$$

against LLL's guaranteed 1.1892.

Lattice. At ML-KEM-512 scale, $d = 512$, $\det(\mathcal{L}) = q^{256}$, $q = 3329$:

$$\det(\mathcal{L})^{1/d} = 3329^{256/512} = \sqrt{3329} \approx 57.70, \quad \text{gh}(\mathcal{L}) \approx \sqrt{\frac{512}{2\pi e}} \cdot 57.70 \approx 5.475 \cdot 57.70 \approx 315.9.$$

Planted vector. As an illustration take a generic LWE error with $\sigma = 3.2$: its norm is about $3.2\sqrt{512} \approx 72.41$, so it is $315.9/72.41 \approx 4.36$ times shorter than a generic vector. ML-KEM-512's actual error is CBD₃, with variance $3/2$ and $\sigma = \sqrt{1.5} \approx 1.22$; its norm is about $1.22\sqrt{512} \approx 27.71$, about 11.40 times shorter than $\text{gh}(\mathcal{L})$.

Published. The estimated attack on ML-KEM-512 needs $\beta \approx 406$, i.e. $0.292 \cdot 406 \approx 118.6$ bits of core-SVP cost.

1.4 Module Lattices

Definition 4 (Coefficient embedding). The coefficient embedding of $R = \mathbb{Z}[X]/(X^n + 1)$ is

$$\iota : R \rightarrow \mathbb{Z}^n, \quad a_0 + a_1X + \dots + a_{n-1}X^{n-1} \mapsto (a_0, \dots, a_{n-1}).$$

The image of an ideal is an **ideal lattice**. Applied coordinatewise, ι sends R^k to $\mathbb{Z}^{nk}$, so a rank- k module is a lattice of dimension $d = nk$.

Proposition 1. *Multiplication by X acts on coefficient vectors as the negacyclic shift*

$$(a_0, \dots, a_{n-1}) \mapsto (-a_{n-1}, a_0, \dots, a_{n-2}),$$

an isometry.

Proof. $X \cdot a_{n-1}X^{n-1} = a_{n-1}X^n = -a_{n-1}$; the map permutes coordinates and flips one sign. $\square$

So one short vector of an ideal lattice gives n independent vectors of the same length. For ML-KEM, $n = 256$ and $k \in \{2, 3, 4\}$, so $d = 512, 768, 1024$.

2. KEMs and the Module Structure

2.1 Key-Encapsulation Mechanisms

Definition 5 (KEM). A key-encapsulation mechanism is a triple of algorithms:

- $\text{KeyGen}() \rightarrow (ek, dk)$: an encapsulation and a decapsulation key;
- $\text{Encaps}(ek) \rightarrow (K, c)$: a shared secret K and a ciphertext c — it takes *no message*;
- $\text{Decaps}(dk, c) \rightarrow K'$.

It is correct if $K' = K$ with overwhelming probability.

The algorithm invents K ; no caller chooses it. For ML-KEM, $K \in \{0, 1\}^{256}$ for every parameter set.

	PKE	KEM
Sender supplies	a message m	nothing
Output	c	(K, c)
Receiver learns	m	K
Example	Regev, ElGamal	ML-KEM

This is what a protocol like TLS needs: a fresh shared key, as Diffie–Hellman gives. ML-KEM replaces Diffie–Hellman, not RSA encryption. Since the caller supplies no message, there is no padding to get wrong. To encrypt a long message one uses **KEM-DEM**: K keys an authenticated symmetric cipher (AES-GCM) for the payload.

2.2 One Ring, Three Parameter Sets

Every parameter set uses

$$n = 256, \quad q = 3329, \quad R_q = \mathbb{Z}_q[X]/(X^{256} + 1).$$

Set	k	η_1	η_2	(d_u, d_v)	Category
ML-KEM-512	2	3	2	(10, 4)	1
ML-KEM-768	3	2	2	(10, 4)	3
ML-KEM-1024	4	2	2	(11, 5)	5

Categories 1, 3 and 5 mean at least as hard to break as AES-128, AES-192 and AES-256. Only the module rank k really varies. Since $3329 < 2^{12}$, a coefficient fits in 12 bits and a ring element serialises to $256 \cdot 12/8 = 384$ bytes.

2.3 The Public Key Is Module-LWE

The objects are

$$\mathbf{A} \in R_q^{k \times k}, \quad \mathbf{s}, \mathbf{e} \in R_q^k, \quad \mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e} \in R_q^k.$$

$\mathbf{A}$ is uniform, expanded from a 32-byte seed ρ with SHAKE128, and never transmitted. The coefficients of $\mathbf{s}$ and $\mathbf{e}$ are drawn from CBD_{η_1} (Lecture 6, §1.4), so $\|\mathbf{s}\|_\infty, \|\mathbf{e}\|_\infty \leq \eta_1 \leq 3$. The pair $(\mathbf{A}, \mathbf{t})$ is exactly an MLWE sample (Lecture 6, §2.3). Without $\mathbf{e}$, $\mathbf{s} = \mathbf{A}^{-1}\mathbf{t}$ by linear algebra whenever $\mathbf{A}$ is invertible.

Example 4 (A small Module-LWE instance). Work in $\mathbb{Z}_{17}[X]/(X^4 + 1)$ with $k = 2$, writing (a_0, a_1, a_2, a_3) for $a_0 + a_1X + a_2X^2 + a_3X^3$. Take

$$\mathbf{A} = \begin{pmatrix} (3, 1, 4, 1) & (5, 9, 2, 6) \\ (2, 7, 1, 8) & (1, 4, 1, 5) \end{pmatrix}, \quad \mathbf{s} = \begin{pmatrix} (1, 0, -1, 1) \\ (0, 1, 1, 0) \end{pmatrix}, \quad \mathbf{e} = \begin{pmatrix} (0, 1, 0, -1) \\ (1, 0, 0, 1) \end{pmatrix}.$$

One product in full. $(3 + X + 4X^2 + X^3)(1 - X^2 + X^3) = 3 + X + X^2 + 3X^3 - 3X^4 + 3X^5 + X^6$. With $X^4 = -1$ this is $(3 + 3) + (1 - 3)X + (1 - 1)X^2 + 3X^3 = (6, -2, 0, 3)$.

All four, centered in $\{-8, \dots, 8\}$:

$$A_{11}s_1 = (6, -2, 0, 3), \quad A_{12}s_2 = (-8, -1, -3, -6), \quad A_{21}s_1 = (-4, -3, 8, 3), \quad A_{22}s_2 = (-6, -4, 5, 5).$$

Adding rows and reducing modulo 17,

$$\mathbf{A}\mathbf{s} = \begin{pmatrix} (-2, -3, -3, -3) \\ (7, -7, -4, 8) \end{pmatrix}, \quad \mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e} = \begin{pmatrix} (-2, -2, -3, -4) \\ (8, -7, -4, -8) \end{pmatrix}.$$

The coefficients of $\mathbf{s}$ and $\mathbf{e}$ lie in $\{-1, 0, 1\}$; those of $\mathbf{t}$ spread over the whole range and look uniform. That is decision MLWE.

2.4 Why k Moves and n Does Not

1. *One NTT.* Changing n changes the two-adicity q must have, hence q and the whole transform. Changing k changes a loop bound.
2. *Granularity.* With power-of-two n the only move is $256 \rightarrow 512$; k moves in steps of 256 lattice dimensions.
3. *Less structure.* A rank- k module has fewer automorphisms than a ring of degree nk , a hedge against structural attacks on Ring-LWE.

3. K-PKE

Inside ML-KEM is an IND-CPA encryption scheme for 256-bit messages, called **K-PKE**. It is never exposed on its own; Section 5 explains why.

3.1 The Algorithms

Definition 6 (K-PKE). Let G be SHA3-512.

- KeyGen: sample a seed d , set $(\rho, \sigma) = G(d\|k)$, expand $\mathbf{A}$ from ρ , sample $\mathbf{s}, \mathbf{e} \leftarrow \text{CBD}_{\eta_1}^k$ from σ , and set $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$. Output $ek_{\text{PKE}} = (\mathbf{t}, \rho)$ and $dk_{\text{PKE}} = \mathbf{s}$.
- Encrypt(ek_{PKE}, m, r), $m \in \{0, 1\}^{256}$: from r sample $\mathbf{y} \leftarrow \text{CBD}_{\eta_1}^k$, $\mathbf{e}_1 \leftarrow \text{CBD}_{\eta_2}^k$ and $e_2 \leftarrow \text{CBD}_{\eta_2}$ (one ring element). Compute

$$\mathbf{u} = \mathbf{A}^\top \mathbf{y} + \mathbf{e}_1, \quad v = \mathbf{t}^\top \mathbf{y} + e_2 + \lfloor q/2 \rfloor m,$$

and output $c = (\text{Compress}_{d_u}(\mathbf{u}), \text{Compress}_{d_v}(v))$.

- Decrypt($\mathbf{s}, c$): decompress to $\mathbf{u}', v'$, compute $w = v' - \mathbf{s}^\top \mathbf{u}'$, and output $\text{Compress}_1(w)$: bit 1 where the centered coefficient is nearer $\pm q/2$ than 0.

Bit m_i becomes the coefficient $\lfloor q/2 \rfloor m_i = 1665 m_i$ of X^i : the scaling factor Δ of Lecture 1 with $t = 2$.

3.2 Correctness

Proposition 2. Write $\mathbf{u}' = \mathbf{u} + \mathbf{c}_u$ and $v' = v + c_v$ for the compression errors. Then

$$w = \lfloor q/2 \rfloor m + \delta, \quad \delta = \mathbf{e}^\top \mathbf{y} - \mathbf{s}^\top \mathbf{e}_1 + e_2 + (c_v - \mathbf{s}^\top \mathbf{c}_u),$$

and $\text{Compress}_1(w) = m$ whenever $\|\delta\|_\infty < q/4$.

Proof. $\mathbf{t}^\top \mathbf{y} = \mathbf{s}^\top \mathbf{A}^\top \mathbf{y} + \mathbf{e}^\top \mathbf{y}$ and $\mathbf{s}^\top \mathbf{u} = \mathbf{s}^\top \mathbf{A}^\top \mathbf{y} + \mathbf{s}^\top \mathbf{e}_1$ (R_q is commutative); the $\mathbf{s}^\top \mathbf{A}^\top \mathbf{y}$ terms cancel in $v - \mathbf{s}^\top \mathbf{u}$, and compression adds $c_v - \mathbf{s}^\top \mathbf{c}_u$. A coefficient $\lfloor q/2 \rfloor m_i + \delta_i$ with $|\delta_i| < q/4$ is nearer $\lfloor q/2 \rfloor m_i$ than the other candidate. $\square$

The cancellation is Diffie–Hellman written additively: the sender never knows $\mathbf{s}$, the receiver never knows $\mathbf{y}$, yet both reach $\mathbf{s}^\top \mathbf{A}^\top \mathbf{y}$. The argument is the correctness template of Lecture 10, with threshold $q/4$ because each coefficient carries one bit.

3.3 The Error Terms

- $\mathbf{e}_1$ hides $\mathbf{A}^\top \mathbf{y}$. It enters δ multiplied by $\mathbf{s}$: $\|\mathbf{s}^\top \mathbf{e}_1\|_\infty \leq kn\eta_1\eta_2$.
- e_2 hides $\mathbf{t}^\top \mathbf{y}$. It enters unmultiplied, so it contributes at most η_2 : nearly free.
- $\mathbf{e}$ from KeyGen returns as $\mathbf{e}^\top \mathbf{y}$ in *every* ciphertext under that key.

For ML-KEM-768 ($k = 3$, $n = 256$, $\eta_1 = \eta_2 = 2$), ignoring compression, the worst case is

$$kn\eta_1^2 + kn\eta_1\eta_2 + \eta_2 = 3072 + 3072 + 2 = 6146 > q/4 = 832.25,$$

a factor of more than seven too large. The bound is absurdly pessimistic: it assumes all kn products take their extreme value with the same sign. A coefficient of $\mathbf{e}^\top \mathbf{y}$ is a sum of kn independent products, and its tail is Gaussian.

3.4 Decryption Failure Probability

FIPS 203 computes the failure probability exactly, by convolving the distributions:

	ML-KEM-512	ML-KEM-768	ML-KEM-1024
failure probability	2^{-139}	2^{-164}	2^{-174}

Acceptable for a KEM. The rate is far below any number of protocol runs, and (Section 5) a failure returns a pseudorandom key, not a visible error.

Not acceptable for FHE. A failure there silently returns a wrong plaintext in the middle of a computation, so BFV and BGV choose parameters that make correctness deterministic.

The rate must still be below about $2^{-\lambda}$: ciphertexts that fail have noise correlated with $\mathbf{s}$, and an attacker who can find them mounts a reaction attack.

Example 5 (K-PKE end to end). Use $n = 4$, $q = 17$, $k = 2$, no compression, and the keys of Example 4. Encrypt $m = (1, 0, 1, 1)$, so $\lfloor q/2 \rfloor m = 9 \cdot (1, 0, 1, 1) = (9, 0, 9, 9)$, with

$$\mathbf{y} = \begin{pmatrix} (1, 1, 0, 0) \\ (0, -1, 1, 0) \end{pmatrix}, \quad \mathbf{e}_1 = \begin{pmatrix} (0, 0, 1, 0) \\ (1, 0, 0, 0) \end{pmatrix}, \quad e_2 = (0, 1, 0, -1).$$

Encrypt. $\mathbf{A}^\top \mathbf{y} = ((-8, -6, 0, -6), (3, 8, 8, -6))$, so

$$\mathbf{u} = \mathbf{A}^\top \mathbf{y} + \mathbf{e}_1 = \begin{pmatrix} (-8, -6, 1, -6) \\ (4, 8, 8, -6) \end{pmatrix} \equiv \begin{pmatrix} (9, 11, 1, 11) \\ (4, 8, 8, 11) \end{pmatrix}.$$

$\mathbf{t}^\top \mathbf{y} = (-2, -4, -7, 7)$, so

$$v = (-2, -4, -7, 7) + (0, 1, 0, -1) + (9, 0, 9, 9) = (7, -3, 2, 15) \equiv (7, 14, 2, 15).$$

Decrypt. $\mathbf{s}^\top \mathbf{u} \equiv (14, 14, 10, 8)$, so $w = v - \mathbf{s}^\top \mathbf{u} = (-7, 0, -8, 7) \equiv (10, 0, 9, 7)$. Subtracting $(9, 0, 9, 9)$ leaves $\delta = (1, 0, 0, -2)$.

Check. $\mathbf{e}^\top \mathbf{y} = (2, -1, 2, -1)$, $\mathbf{s}^\top \mathbf{e}_1 = (1, 0, 2, 0)$, and $\mathbf{e}^\top \mathbf{y} - \mathbf{s}^\top \mathbf{e}_1 + e_2 = (1, 0, 0, -2) = \delta \checkmark$. Since $\|\delta\|_\infty = 2 < q/4 = 4.25$, rounding the centered $w = (-7, 0, -8, 7)$ gives $(1, 0, 1, 1) = m \checkmark$.

4. Compress and Decompress

4.1 Definition and Error

Definition 7 (Compress, Decompress). For $d < \lceil \log_2 q \rceil$ and $x \in \mathbb{Z}_q$,

$$\text{Compress}_d(x) = \left\lfloor \frac{2^d}{q} x \right\rfloor \bmod 2^d, \quad \text{Decompress}_d(y) = \left\lfloor \frac{q}{2^d} y \right\rfloor,$$

applied coefficientwise to elements of R_q and R_q^k .

Rounding (Lecture 1) keeps the error centered at zero, so it does not accumulate across the kn terms of $\mathbf{s}^\top \mathbf{c}_u$.

Proposition 3. For every $x \in \mathbb{Z}_q$, $|\text{Decompress}_d(\text{Compress}_d(x)) - x| \leq B_d := \lfloor q/2^{d+1} \rfloor$, the difference taken as a centered representative modulo q .

Proof sketch. Compress_d rounds $2^d x/q$, an error of at most $1/2$, which Decompress_d scales back to at most $q/2^{d+1}$; the final rounding adds at most $1/2$, and an integer at most $q/2^{d+1} + 1/2$ is at most $\lfloor q/2^{d+1} \rfloor$. $\square$

4.2 Why d_u Is Large and d_v Is Small

For $q = 3329$ the bound is attained:

d	B_d	Used for
1	832	the message: $\text{Decompress}_1(m) = 1665 m$
4	104	d_v , ML-KEM-512 and -768
5	52	d_v , ML-KEM-1024
10	2	d_u , ML-KEM-512 and -768
11	1	d_u , ML-KEM-1024

$\mathbf{c}_u$ is multiplied by $\mathbf{s}$, picking up the factor kn , so $\mathbf{u}$ is compressed gently. c_v enters δ unmultiplied and $104 \ll 832.25$, so v is compressed hard. This is the same asymmetry as $\mathbf{e}_1$ against e_2 .

4.3 The Ciphertext Size

Proposition 4. $|c| = 32 d_u k + 32 d_v = 32 (d_u k + d_v)$ bytes.

Proof. A ring element at d bits per coefficient is $256d/8 = 32d$ bytes; $\mathbf{u}$ is k elements at d_u bits and v one at d_v bits. $\square$

For ML-KEM-768, uncompressed transmission takes $384 \cdot 3 + 384 = 1536$ bytes; compressed it is 1088, a saving of 448 bytes (29%). Compression changes the scheme: its errors appear in δ , and the security analysis is stated for the compressed ciphertext.

5. CCA Security and Fujisaki–Okamoto

5.1 K-PKE Is Malleable

Adding $\text{Decompress}_1(m')$ to v yields a ciphertext of $m \oplus m'$. With a decryption oracle, one query on the modified challenge recovers m . The threat is real: a TLS server decapsulates whatever it is sent and behaves observably afterwards, and Bleichenbacher (1998) broke RSA PKCS#1 v1.5 using only a valid/invalid padding signal. Homomorphic schemes are malleable and cannot be CCA2 secure (Lecture 8); ML-KEM gives up malleability to get CCA2.

5.2 IND-CCA2 for a KEM

Definition 8 (IND-CCA2 for KEMs).

1. The challenger runs $(ek, dk) \leftarrow \text{KeyGen}()$ and gives ek to the attacker.
2. The challenger computes $(K_0, c^*) \leftarrow \text{Encaps}(ek)$, samples K_1 uniformly, picks a random bit b , and sends (c^*, K_b) .
3. Before and after step 2, the attacker may submit any c (after step 2, any $c \neq c^*$) and receives $\text{Decaps}(dk, c)$.
4. The attacker outputs a guess b' .

The KEM is IND-CCA2 secure if for every efficient attacker $|\Pr[b' = b] - \frac{1}{2}| \leq \text{negl}(\lambda)$.

There are no chosen messages: the attacker only has to tell the real K from a random one.

5.3 The Fujisaki–Okamoto Transform

The FO transform turns an IND-CPA PKE into an IND-CCA2 KEM in the random oracle model, by two moves.

1. *Derandomise.* Derive the encryption randomness from the message, $r = G(m)$. Anyone holding m can recompute the unique ciphertext.
2. *Re-encrypt on decryption.* Decrypt to m' , re-encrypt with $G(m')$, and compare with the received ciphertext. Any modification changes m' , hence the re-encryption, and is caught.

Definition 9 (Random oracle model). A hash function is modelled as a truly random function, available only through an oracle: each fresh input gets an independent uniform output, and the security proof may see and answer every query.

No real hash function is a random oracle, so the proof is evidence, not a theorem about the deployed system.

5.4 ML-KEM

Let $H = \text{SHA3-256}$, $G = \text{SHA3-512}$ and $J = \text{SHAKE256}$ (32-byte output).

Definition 10 (ML-KEM).

- KeyGen: run K-PKE.KeyGen and sample a 32-byte **rejection key** z . Output $ek = ek_{\text{PKE}}$ and $dk = dk_{\text{PKE}} \| ek_{\text{PKE}} \| H(ek_{\text{PKE}}) \| z$.
- Encaps(ek):
 1. $m \leftarrow \{0, 1\}^{256}$;
 2. $(K, r) = G(m \| H(ek))$;
 3. $c = \text{K-PKE.Encrypt}(ek, m, r)$; output (K, c) .
- Decaps(dk, c):
 1. $m' = \text{K-PKE.Decrypt}(dk_{\text{PKE}}, c)$;
 2. $(K', r') = G(m' \| H(ek_{\text{PKE}}))$;
 3. $\bar{K} = J(z \| c)$;
 4. $c' = \text{K-PKE.Encrypt}(ek_{\text{PKE}}, m', r')$;
 5. if $c' \neq c$, set $K' = \bar{K}$;
 6. output K' .

One call to G gives both K and r , the two 32-byte halves of its output. Mixing in $H(ek)$ binds K to the public key. dk stores ek_{PKE} , $H(ek_{\text{PKE}})$ and z only so that the check needs no other input. Decaps runs a full encryption, so it is slower than Encaps.

5.5 Implicit Rejection

Step 5 returns $\bar{K} = J(z \| c)$, never an error.

- *No oracle.* A rejected ciphertext yields a key that looks valid; the protocol fails later, when authentication fails, identically for every bad input.
- *Deterministic.* $\bar{K}$ depends only on z and c , so replaying a bad ciphertext gives the same answer.
- *Absorbs decryption failures.* A rare failure ($m' \neq m$) fails the re-encryption check and also returns $\bar{K}$; the attacker cannot tell why.

The comparison $c' \neq c$ must be **constant time**: an early-exit comparison reintroduces the oracle. The security proof reduces IND-CCA2 of ML-KEM to IND-CPA of K-PKE in the random oracle model, with a loss that grows with the decryption-failure probability — another reason that probability must be tiny.

6. The NTT in FIPS 203

A negacyclic NTT of length n needs a primitive $2n$ -th root of unity (Lecture 3); for $n = 256$, a 512-th root. But

$$q - 1 = 3328 = 2^8 \cdot 13,$$

so $256 \mid q - 1$ ($\zeta = 17$ has order 256: $17^{128} \equiv -1$) while $512 \nmid q - 1$. The transform runs 7 layers and stops at an incomplete splitting:

$$\mathbb{Z}_{3329}[X]/(X^{256} + 1) \cong \prod_{i=0}^{127} \mathbb{Z}_{3329}[X]/(X^2 - \zeta^{2 \text{brv}(i)+1}),$$

where brv is 7-bit bit reversal. There are 128 quadratic factors, not 256 linear ones, and the “pointwise” product is **BaseCaseMultiply**: multiplying degree-1 polynomials modulo $X^2 - \gamma$, as $(a_0 + a_1X)(b_0 + b_1X) = (a_0b_0 + \gamma a_1b_1) + (a_0b_1 + a_1b_0)X$.

FIPS 203 defines its arithmetic in the NTT domain: $\hat{\mathbf{A}}$ is sampled there directly from ρ , and ek stores $\hat{\mathbf{t}} = \text{NTT}(\mathbf{t})$.

Example 6 (Counting the layers). ML-KEM uses $q = 3329$; ML-DSA (Lecture 18) uses $q = 8380417$.

Factor. $3328 = 2^8 \cdot 13$, two-adicity 8; $8380416 = 2^{13} \cdot 1023$, two-adicity 13.

Layers. A length-256 negacyclic NTT needs $512 = 2^9 \mid q - 1$ to run all 8 layers. ML-KEM: $2^9 \nmid 3328$, so 7 layers. ML-DSA: $2^9 \mid 8380416$, so all 8.

Factors. ML-KEM: 128 factors of degree 2. ML-DSA: 256 factors of degree 1.

Pointwise cost. ML-KEM: $a_0b_0, a_1b_1, a_0b_1, a_1b_0$ and one multiplication by γ , i.e. 5 multiplications per factor. ML-DSA: one multiplication per factor.

7. Sizes and Deployment

7.1 Sizes

Proposition 5. In bytes, $ek = 384k + 32$, $dk = 768k + 96$, $c = 32(d_uk + d_v)$.

Proof. ek is $\hat{\mathbf{t}}$ ($384k$) and ρ (32) — not $384k^2$ for $\mathbf{A}$. dk is $\hat{\mathbf{s}}$ ($384k$), ek ($384k + 32$), $H(ek)$ (32) and z (32). c is Proposition 4. $\square$

Example 7 (Deriving the sizes). With $(k, d_u, d_v) = (2, 10, 4), (3, 10, 4), (4, 11, 5)$:

Set	$ek = 384k + 32$	$dk = 768k + 96$	$c = 32(d_uk + d_v)$
ML-KEM-512	$768 + 32 = 800$	$1536 + 96 = 1632$	$32 \cdot 24 = 768$
ML-KEM-768	$1152 + 32 = 1184$	$2304 + 96 = 2400$	$32 \cdot 34 = 1088$
ML-KEM-1024	$1536 + 32 = 1568$	$3072 + 96 = 3168$	$32 \cdot 49 = 1568$

For ML-KEM-768, $\mathbf{u}$ takes $32 \cdot 10 \cdot 3 = 960$ bytes and v takes $32 \cdot 4 = 128$: $\mathbf{u}$ is 88% of the ciphertext.

If $\mathbf{A}$ were sent instead of ρ , ek would grow by $384k^2 - 32$ bytes: 1504, 3424 and 6112 for $k = 2, 3, 4$. Since dk stores a copy of ek , it would grow by the same amount; c would not change.

7.2 Deployment: Hybrid X25519MLKEM768

In TLS 1.3 the client's `key_share` is ek and the server's is c . ML-KEM is deployed as a hybrid with X25519:

Message	Contents	Bytes
Client	ek (1184) + X25519 (32)	1216
Server	c (1088) + X25519 (32)	1120
Total	(X25519 alone: 64)	2336

The two shared secrets are concatenated and fed to the key schedule, so the result is secure if *either* half holds. The reasons are harvest-now-decrypt-later, and the 2022 breaks of Rainbow and SIKE (Lecture 16), which show that years of analysis are no guarantee. ML-KEM gives confidentiality only; authentication needs a post-quantum signature, ML-DSA (Lecture 18).

Summary

1. Parameters come from lattice reduction: they are chosen so that the BKZ block size needed to find the planted vector costs more than 2^λ .
2. ML-KEM fixes $n = 256$, $q = 3329$ and varies the rank $k \in \{2, 3, 4\}$; the public key $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$ is an MLWE instance.
3. K-PKE decrypts to $\lfloor q/2 \rfloor m + \delta$ and is correct when $\|\delta\|_\infty < q/4$; the failure probability is at most 2^{-139} , fine for a KEM but not for FHE.
4. Compression gives $|c| = 32(d_u k + d_v)$; d_u is large because $\mathbf{c}_u$ is multiplied by $\mathbf{s}$.
5. The Fujisaki–Okamoto transform with implicit rejection lifts IND-CPA to IND-CCA2; the re-encryption check must be constant time.
6. $q - 1 = 2^8 \cdot 13$, so the NTT stops at 128 quadratic factors.

Exercises

1. Run Lagrange–Gauss on the basis $b_1 = (1, 1)$, $b_2 = (3, 4)$ of $\mathbb{Z}^2$, showing μ and $\lfloor \mu \rfloor$ at each step. Compute the orthogonality defect before and after.
2. Evaluate $\delta_0(\beta)$ for $\beta = 200$ and $\beta = 300$ and the corresponding classical and quantum core-SVP costs. Which β gives 128 bits of quantum security?
3. Verify the remaining three products $A_{ij}s_j$ of Example 4 by hand, and confirm $\mathbf{t} - \mathbf{A}\mathbf{s} = \mathbf{e}$.
4. In Example 5, change e_2 to $(4, 4, 4, 4)$. Recompute v , w and δ , and determine which message bits decrypt incorrectly.
5. If ML-KEM-768 kept $d_u = 10$ but used $d_v = 3$, how many bytes would the ciphertext save, and how much would the worst-case contribution of c_v to δ grow?
6. Compute 17^{128} and 17^{256} modulo 3329 by repeated squaring. Explain why $17^{128} \equiv -1$ is what is required, and show that no element of $\mathbb{Z}_{3329}^\times$ has order 512.

7. An attacker adds $\text{Decompress}_1(m')$, $m' \neq 0$, to v in a challenge ciphertext and submits it to Decaps. Trace each step and identify where the attack fails. Then suppose step 5 returned $\perp$ instead of $\bar{K}$: what oracle does the attacker obtain?